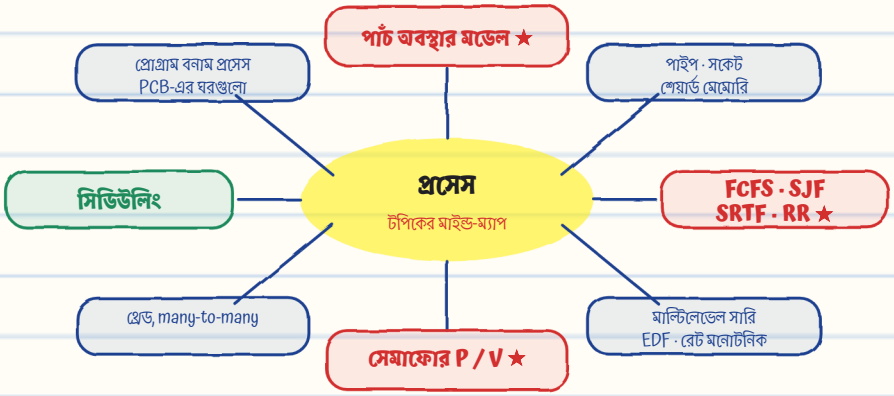
তারিখ: ০৯.১০.২০২৬
✎ টপার নোটস

প্রক্রিয়া ও সিঙ্ক্রোনাইজেশন

PCB · পাঁচ অবস্থা · IPC ও সেমাফোর · CPU সিঙ্ক্রোনাইজেশন: ১৩ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = যে ধারণার ওপর সিঙ্ক্রোনাইজেশনের সব মাপ দাঁড়ায়

➜ সিলেবাসের যে অংশ

সিলেবাসে আছে "*Process: process model and implementation, Inter-Process Communication (IPC), classical IPC problems, process scheduling, multiprocessing and time-sharing*"। ডেডলক, মেমোরি ম্যানেজমেন্ট ও ফাইল সিস্টেমের নিজস্ব লেসন আছে।

পাশের লেসনগুলো। রিডার্স-রাইটার্স, ডাইনিং ফিলোসফার্স ও স্লিপিং বার্বার এই আদর্শগুলোর ওপর দাঁড়ানো সিঙ্ক্রোনাইজেশন সমস্যা; ডেডলক প্রতিরোধ, ব্যাংকার অ্যালগরিদম ও পুনরুদ্ধার আলাদা লেসনে।

হাই-ইল্ড তালিকা

① প্রোগ্রাম = ফাইল · প্রসেস = মেমোরিতে
② PCB: PID, state, PC, রেজিস্টার, শিডিউলিং, মেমোরি, অ্যাকাউন্ট
③ new → ready → running → waiting → terminated
④ রেডি CPU চায় · ওয়েটিং কোনো ঘটনা চায়
⑤ দীর্ঘ-, মধ্যবর্তী-, স্বল্পমেয়াদি শিডিউলার
⑥ থ্রেডরা অ্যাড্রেস স্পেস ভাগ করে
⑦ IPC: shm, pipe, FIFO, socket, signal
⑧ সেমাফোর P = −1, V = +1
⑨ TAT = C − অ্যারাইভাল · WT = TAT − বার্স্ট
⑩ SJF সেরা গড় অপেক্ষা · RR = q পালা ★

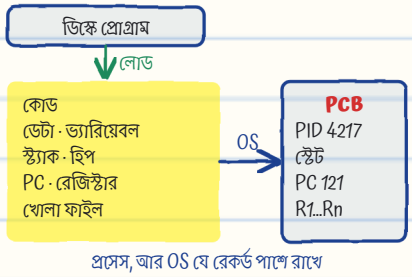
সংকেত হলুদ = মূল শব্দ · লাল বৃত্ত = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

➜ সূচিপত্র

✓ প্রোগ্রাম, প্রসেস ও PCB ... পৃ. ২
✓ পাঁচ অবস্থা ও সারিগুলো ... পৃ. ৩
✓ তৈরি ও সমাপ্তি ... পৃ. ৪
✓ থ্রেড ও তার মডেল ... পৃ. ৫
✓ IPC-এর পদ্ধতি ... পৃ. ৬
✓ সেমাফোর, প্রোডিউসার-কনজিউমার ... পৃ. ৭
✓ সিঙ্ক্রোনাইজেশনের মাপ ও প্রিম্পশন ... পৃ. ৮
✓ FCFS, SJF, প্রায়োরিটি ... পৃ. ৯
✓ SRTF ও রাউন্ড রবিন ... পৃ. ১০
✓ মাল্টিলেভেল সারি, EDF ... পৃ. ১১
✓ রাতের পাতা পৃ. ১২ · অনুশীলন পৃ. ১৩

১ প্রোগ্রাম বনাম প্রসেস ও PCB সংজ্ঞা ★

সংজ্ঞা একটি **প্রোগ্রাম** হলো ডিস্কে রাখা নির্দেশনার নিষ্ক্রিয় ফাইল। একটি **প্রসেস** হলো সেই প্রোগ্রাম **মেমোরিতে তুলে চালু হওয়া** — তার কোড, ডেটা, প্রোগ্রাম কাউন্টার, সে যে রেজিস্টারগুলো ধরে আছে এবং খোলা ফাইলসহ। **প্রোগ্রাম স্থির, প্রসেস গতিশীল** — একটি প্রোগ্রাম একসঙ্গে অনেক প্রসেস হতে পারে।



একটি প্রোগ্রাম, একটি ইনস্ট্যান্স

বিষয়	প্রোগ্রাম	প্রসেস
প্রকৃতি	নিষ্ক্রিয়, স্থির	সক্রিয়, গতিশীল
মেমোরি	ডিস্কই যথেষ্ট	মেইন মেমোরি লাগে
পরিচয়	একটি ফাইলের নাম	PID, অনেক ইনস্ট্যান্স
জীবন	যতদিন ফাইল আছে	*new → running → terminated*

PCB-র ঘর	কী থাকে	OS-র কেন দরকার
প্রসেস স্টেট	*New · Ready · Running · Waiting · Terminated*	ঠিক করে **কোন সারিতে** আছে
প্রোগ্রাম কাউন্টার	পরবর্তী নির্দেশনার ঠিকানা	**ঠিক যেখানে** থেমেছিল সেখান থেকেই চালু করতে
CPU রেজিস্টার	অ্যাকুমুলেটর, *base, limit*, স্ট্যাক, *R0...Rn*	অসমাপ্ত হিসাব মনে রাখা
CPU শিডিউলিং তথ্য	অগ্রাধিকার, সারির পয়েন্টার	পরে কে চলবে ঠিক করা
মেমোরি ম্যানেজমেন্ট তথ্য	পেজ টেবিল বেস, সেগমেন্ট টেবিল পয়েন্টার	ঠিকানা অনুবাদ
অ্যাকাউন্টিং তথ্য	ইউজার ও কার্নেল সময়, *I/O* গণনা, শিশুর *PID*	হিসাব, কোটা, পরিষ্কার

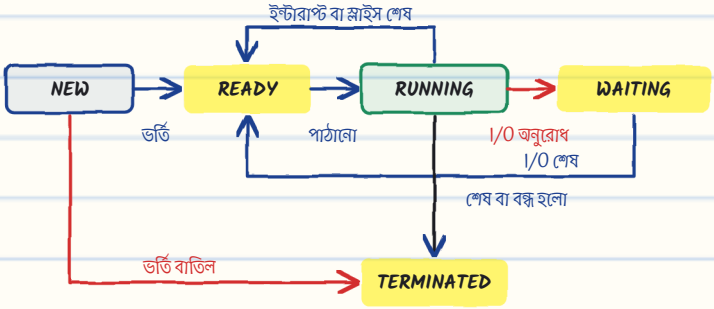
মনে রাখার কৌশল *PCB* হলো প্রসেসের **পাসপোর্ট**: *PID* বলে সে কে, **স্টেট** বলে কোথায়, **PC ও রেজিস্টার** মনে রাখে ঠিক কোথায় থেমেছিল; বাকিগুলো বলে সার কী ধরে আছে।

পরীক্ষার ফাঁদ *(i) PCB* একটি **কার্নেল** কাঠামো, *OS*-ই বানায়; প্রসেস কখনো তার ভেতরে লেখে না। *(ii)* প্রসেস **তার PCB নয়** — শেষ হলে দুটোই মুক্তি পায়। *(iii) PID* চলমান প্রসেসের মধ্যে অনন্য, পরে **পুনরায় ব্যবহৃত** হতে পারে।

সমাধানকৃত উদাহরণ ✍ একটি PCB ডাম্প পড়ো *PID 4217*, স্টেট ***Waiting***, *PC 121, base 240, limit 460*, ইউজার সময় *128*, কার্নেল সময় *12*. **(ক)** এটি রেডি সারিতে **নয়** — এটি একটি *I/O* ঘটনার অপেক্ষায়। **(খ)** বৈধ ঠিকানা = *240 ... 459*। **(গ)** ব্যবহৃত *CPU = 128 + 12 =* *140* ইউনিট। **(ঘ)** *PID* নিজে শিডিউলিংয়ের চাবিকাঠি নয়; **অগ্রাধিকার** হলো।

২ পাঁচ-অবস্থার মডেল ও সারিগুলো সবচেয়ে বেশি আসে ★

সংজ্ঞা একটি প্রসেস ঠিক **পাঁচটি** অবস্থার একটিতে থাকে: *new* (তৈরি হচ্ছে), *ready* (মেমোরিতে, CPU-এর অপেক্ষায়), *running* (CPU ধরে আছে), *waiting / blocked* (CPU ছাড়া অন্য কোনো ঘটনার অপেক্ষায়) এবং *terminated* (শেষ, রিসোর্স মুক্ত)।



সাতটি রূপান্তর; CPU শিডিউলার শুধু **রেডি** থেকেই বাছাই করে

থেকে → পর্যন্ত	কী ঘটে	কে করে
New → Ready	ভর্তি	দীর্ঘমেয়াদি শিডিউলার
Ready → Running	CPU বরাদ্দ	স্বল্পমেয়াদি শিডিউলার
Running → Ready	ইন্টারাপ্ট, স্লাইস শেষ, বেশি অগ্রাধিকারের প্রসেস এল	স্বল্পমেয়াদি শিডিউলার
Running → Waiting	I/O অনুরোধ বা ঘটনার অপেক্ষা	প্রসেস নিজে
Waiting → Ready	I/O শেষ, ঘটনা ঘটেছে	ইন্টারাপ্ট হ্যান্ডলার
Running → Terminated	শেষ, ভুল, বা বন্ধ করা হয়েছে	প্রসেস নিজে
New → Terminated	ভর্তি বাতিল	দীর্ঘমেয়াদি শিডিউলার

সারি	কী রাখে / কে খালি করে
জব সারি	এখনো ভর্তি হয়নি; দীর্ঘমেয়াদি শিডিউলার
রেডি সারি	মেমোরিতে CPU-এর অপেক্ষায়; স্বল্পমেয়াদি শিডিউলার
ডিভাইস সারি	ওই I/O-এর অপেক্ষায়; ডিভাইস কন্ট্রোলার

পরীক্ষার ফাঁদ তিনটি শিডিউলার আছে — **দীর্ঘমেয়াদি** (জব সারি), **মধ্যবর্তী** (প্রসেস বাইরে-ভেতরে সোয়াপ) ও **স্বল্পমেয়াদি** (CPU)। শুধু CPU শিডিউলারটিই অবিরাম চলে।

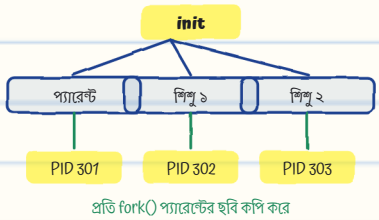
মনে রাখার কৌশল "**রেডি** চায় CPU, **ওয়েটিং** চায় **অন্য কিছু**।" দুটোই একইভাবে ফেরে — ঘটনা শেষ, প্রসেস আবার **রেডি** সারিতে যায়, সরাসরি CPU-তে নয়।

পরীক্ষার ফাঁদ শুধু CPU-এর অপেক্ষা মানে **রেডি**; ওয়েটিং নয় — এটিই সবচেয়ে বেশি জিজ্ঞাসিত অবস্থা-প্রশ্ন। ডিভাইস সারিতে থাকা প্রসেস মেমোরিতে থেকেও ওয়েটিং।

সমাধানকৃত উদাহরণ ✍ একটি প্রসেসের পথ ধরো আপনি একটি কমান্ড লিখলেন। **(ক)** দীর্ঘমেয়াদি শিডিউলার ভর্তি করলে *new → ready*। **(খ)** CPU শিডিউলার পাঠালে *ready → running*। **(গ)** প্রিন্টের অনুরোধ দিলেই *running → waiting*। **(ঘ)** প্রিন্টার শেষ করলে *waiting → ready*। **(ঙ)** আগে ঘড়ির ইন্টারাপ্টে স্লাইস শেষ হয়েছিল: এটি *running → ready*, ***running → waiting* নয়**।

৩ প্রসেস তৈরি ও সমাপ্তি লিখিত প্রশ্ন ★

সংজ্ঞা **তৈরি:** OS একটি PID নেয়, PCB বরাদ্দ করে পূরণ করে, অ্যাড্রেস স্পেস বানায়, খোলা ফাইল জুড়ে দেয় এবং PCB-কে রেডি সারিতে রাখে। **সমাপ্তি:** স্টেট টার্মিনেটেড হয়, মেমোরি ও ফাইল মুক্ত হয়, এক্সিট স্ট্যাটাস প্যারেন্টের কাছে যায়।



প্রতিটি শিশুর ঠিক **একজন** প্যারেন্ট থাকে

তৈরির ধাপ	যা ঘটে
১. PID নাও	PID টেবিল থেকে; মুক্তির পর পুনর্ব্যবহার
২. PCB বরাদ্দ	কন্ট্রোল ব্লকের জন্য মেমোরি সংরক্ষিত
৩. প্রাথমিকায়ন	স্টেট, **PC এন্ট্রি পয়েন্টে**, অ্যাকাউন্ট = 0, সারির পয়েন্টার
৪. অ্যাড্রেস স্পেস	প্যারেন্টের ছবি কপি; পেজ/সেগমেন্ট টেবিল তৈরি
৫. খোলা ফাইল	ডেস্ক্রিপ্টর একই ইনোডের দিকে যায়
৬. রিসোর্স	I/O, মেমোরি ও CPU কোটা
৭. কনটেক্সট প্রস্তুত	প্রথম পাঠানোর জন্য স্ট্যাক ও রেজিস্টার
৮. সারিতে ঢোকা	PCB → রেডি সারি; কখনো সরাসরি running নয়

প্রসেস শেষ হওয়ার উপায়	সাধারণ কারণ
স্বাভাবিক শেষ	main() ফেরত হয় বা exit(0); মানটিই এক্সিট স্ট্যাটাস
ভুলে শেষ	ফ্যাটাল এরর — শূন্যে ভাগ, অবৈধ মেমোরি রেফারেন্স
অন্য প্রসেস বন্ধ করে	SIGKILL-এর মতো সিগন্যাল, যা ধরা যায় না
OS বন্ধ করে	মেমোরি সীমা অতিক্রম, প্রোটেকশন ফল্ট, শাটডাউন

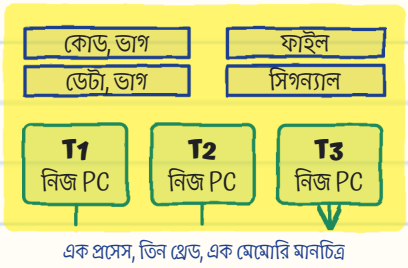
পরীক্ষার ফাঁদ প্যারেন্ট তার খবর না নেওয়া পর্যন্ত শেষ হয়ে যাওয়া শিশুকে **জম্বি** বলে: কাজ শেষ, কিন্তু এক্সিট স্ট্যাটাস ও কিছু টেবিল এন্ট্রি প্যারেন্ট অপেক্ষা করে। প্যারেন্ট আগে মারা গেলে শিশু হয় **অরফান**, *init* তাকে গ্রহণ করে।

মনে রাখার কৌশল **"জন্ম রেডি, মৃত্যু শেষ।"** তৈরি শেষ হয় রেডি সারিতে; সমাপ্তিতে PID ও PCB মুক্তি পায়। *init* অরফান গ্রহণ করে বলে কোনো জম্বি অসংগৃহীত থাকে না।

সমাধানকৃত উদাহরণ ✍ একটি শেল সেশন আপনি লিখলেন `ls > out.txt`। **(ক)** শেল দুইবার *fork* করে — একবার `ls`-এর জন্য, একবার রিডাইরেকশনের ডান পাশের জন্য। **(খ)** `exec`-এর আগে শিশু ফাইলটি খোলে; ইনোড **ভাগ** হয়, কপি নয়। **(গ)** `exec` ছবি বদলায় কিন্তু **PID রাখে**। **(ঘ)** `ls` শেষ হলে শেলের `wait()` তাকে সংগ্রহ করে, তাই জম্বি মিলিসেকেন্ডই থাকে।

৪ থ্রেড: এক অ্যাড্রেস স্পেসের ভাগ হাই-ইল্ড ★

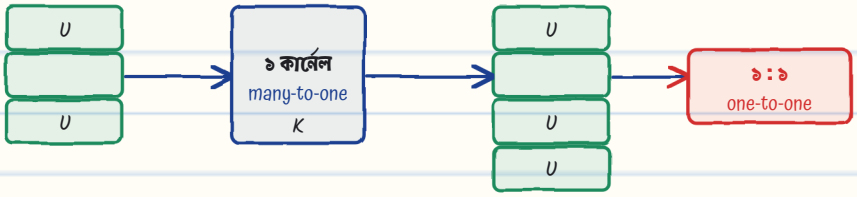
সংজ্ঞা একটি থ্রেড হলো প্রসেসের ভেতরে **কার্যকরণের একক**। একই প্রসেসের থ্রেডগুলো **অ্যাড্রেস স্পেস, কোড, ডেটা, খোলা ফাইল ও সিগন্যাল ভাগ করে**; প্রতিটির নিজস্ব **স্ট্যাক, রেজিস্টার, প্রোগ্রাম কাউন্টার ও থ্রেড-লোকাল স্টোরেজ** থাকে।



এক প্রসেস, তিন থ্রেড, এক মেমোরি মানচিত্র

নিজস্ব স্ট্যাক, এক অ্যাড্রেস স্পেস

বিষয়	প্রসেস	থ্রেড
অ্যাড্রেস স্পেস	নিজস্ব, সুরক্ষিত	ভাইদের সাথে ভাগ
নিজস্ব অবস্থা	সবকিছু	স্ট্যাক, রেজিস্টার, *PC*, *TLS*
তৈরির খরচ	ভারী: ছবি কপি হয়	সস্তা: একটি *TCB*
যোগাযোগ	*IPC* লাগে	শেয়ার্ড ভ্যারিয়েবল পড়লেই হলো
ব্যর্থতার প্রভাব	সীমাবদ্ধ	খারাপ লেখা **সব** থ্রেড মারতে পারে
শিডিউলারে	সবসময় আলাদা সত্তা	নিচের মডেলের ওপর নির্ভর করে



many-to-many: m ইউজার থ্রেড n কার্নেল থ্রেডে, m ≥ n

তিনটি মাল্টিথ্রেডেড মডেল – *OS* সবসময় **কার্নেল** থ্রেড শিডিউল করে

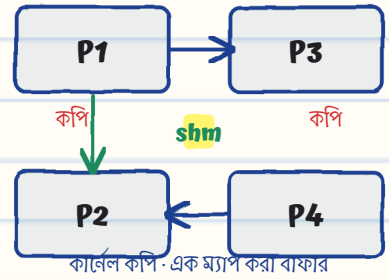
স্তর	কোথায় থাকে	যা দেয়
ইউজার-লেভেল	প্রসেসের ভেতরের একটি লাইব্রেরি	সস্তা সুইচ, কিন্তু একটি ব্লকিং কল পুরো **প্রসেসকেই** থামায়
কার্নেল-লেভেল	*OS* প্রতিটি সত্তার জন্য আসল থ্রেড রাখে	একটি ব্লকিং কল **একটি** থ্রেড থামায়; অনেক কোরে সত্যিকারের সমান্তরালতা

পরীক্ষার ফাঁদ ***many-to-one***-এ ব্লকিং কল **সব** থ্রেড থামায় এবং কোনো সমান্তরালতা নেই; ***one-to-one***-এ সমান্তরালতা আসল, কিন্তু প্রতিটি থ্রেডের জন্য *PCB* ও কার্নেল স্ট্যাক লাগে। প্রথম দুই মডেলে **থ্রেড শিডিউলিং ≠ প্রসেস শিডিউলিং**: কার্নেল কার্নেল থ্রেড শিডিউল করে, লাইব্রেরি ইউজার থ্রেড।

মনে রাখার কৌশল **"থ্রেড ভাগ করে, কপি করে না।"** `fork()` বলতে হলে সেটি **প্রসেস**; শুধু একটি গ্লোবালে মান বসালেই হলো সেটি **থ্রেড**।

৫ ইন্টার-প্রসেস কমিউনিকেশন বেশি আসে ★

সংজ্ঞা IPC একটি প্রসেসকে অন্যকে **তথ্য পাঠাতে** এবং তার সাথে **সমন্বয়** করতে দেয়। দুই পরিবার: **মেসেজ পাসিং** (কপি কার্নেল দিয়ে যায়) এবং **শেয়ার্ড মেমোরি** (সবচেয়ে দ্রুত — কোনো কপি নেই — তবে এতে **কোনো** সিঙ্ক্রোনাইজেশন নেই)।



মেসেজ পাসিং বনাম শেয়ার্ড মেমোরি

পদ্ধতি	ধরন	নোট
শেয়ার্ড মেমোরি	*shared*	সবচেয়ে দ্রুত; **লক নিজেই লিখতে হয়**
অ্যানোনিমাস পাইপ	*message*	বাইট স্ট্রিম, কেবল **সম্পর্কিত** প্রসেস
নেমড পাইপ (FIFO)	*message*	ফাইলসিস্টেমের নাম আছে, **অসম্পর্কিত** প্রসেসও খুলতে পারে
মেসেজ কিউ	*message*	কার্নেলে **বিভিন্ন** বার্তার তালিকা
সকেট	*message*	নামযুক্ত প্রান্ত, স্থানীয় **বা** *TCP/IP*
সিগন্যাল	*message*	একপাশে **সংকেত**; *SIGKILL* ধরা যায় না
মেইলবক্স	*message*	পরোক্ষ: নামযুক্ত ড্রপ বক্স

কখন বেছে নেবেন	খরচ
শেয়ার্ড মেমোরি বড় বাফারের জন্য — ভিডিও ফ্রেম	প্রতিটি লক আপনাকেই লিখতে হবে
পাইপ / *FIFO* বাইটের ধারা ছাঁকতে	বাফার সীমিত, বার্তার সীমানা নেই
মেসেজ কিউ ক্রম ঠিক রেখে টুকরো কাজের জন্য	কিউয়ের নির্দিষ্ট সর্বোচ্চ দৈর্ঘ্য
সকেট অন্য মেশিন বা সার্ভিসে পৌঁছাতে	সিরিয়ালাইজেশন, প্রতি কানেকশনে আলাদা অ্যাড্রেস স্পেস
সিগন্যাল "কিছু বদলেছে — আবার পড়ো" বলতে	কোনো তথ্য নেই; হ্যান্ডলার অস্বাভাবিক সময়ে চলে

পরীক্ষার ফাঁদ **পাইপ কেবল সম্পর্কিত প্রসেসের জন্য** — একটি শেল পাইপলাইন, বা প্যারেন্ট ও তার শিশু। দুটি অসম্পর্কিত প্রোগ্রামের জন্য লাগবে **নেমড *FIFO***, সকেট বা শেয়ার্ড মেমোরি। আর শেয়ার্ড মেমোরি কোনো **মিউচুয়াল এক্সক্লুশন** দেয় না: দুই লেখক দৌড়ে গেলে ডেটা হারায়।

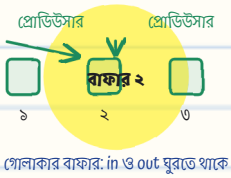
মনে রাখার কৌশল "**কপি** মানে মেসেজ পাসিং, **ম্যাপ** মানে শেয়ার্ড মেমোরি, **নামযুক্ত** মানে অসম্পর্কিত।" ডেটা যদি নড়েই না, তবে নিরাপদ করার কাজ মিউটেক্সের।

সমাধানকৃত উদাহরণ ✍ পদ্ধতি বাছো **(ক)** `sort < file | head` — অ্যানোনিমাস **পাইপ**। **(খ)** একটি লগার ডেমন ও ৪০টি অসম্পর্কিত সার্ভিস — **নেমড *FIFO*** বা **সকেট**। **(গ)** দুই ডিকোডারে ৪ *MB* ফ্রেম লেখা একটি ভিডিও সার্ভার — **শেয়ার্ড মেমোরি** প্লাস মিউটেক্স। **(ঘ)** "কনফিগ ফাইল বদলেছে" — একটি **সিগন্যাল**, তারপর প্রসেস নিজে ফাইল আবার পড়ে: সিগন্যাল একটা ঠেলা, স্থানান্তর নয়।

৬ সেমাফোর, মিউটেক্স ও প্রোডিউসার-কনজিউমার

লিখিত প্রশ্ন ★

সংজ্ঞা একটি **সেমাফোর** (ডিকস্ট্রা, ১৯৬৫) হলো ঋণাত্মক নয় এমন একটি পূর্ণসংখ্যা ও একটি অপেক্ষা-সারি, যা কেবল দুটি **অ্যাটমিক** ক্রিয়ায় বদলায়: **P (ওয়েট):** $S = 0$ হলে ব্লক, তারপর $S \leftarrow S - 1$ এবং **V (সিগন্যাল):** $S \leftarrow S + 1$, একজন অপেক্ষাকে জাগান। **বাইনারি** সেমাফোর — মিউটেক্স — 0 বা 1-এই থাকে।



গোলাকার বাফার: in ও out ঘুরতে থাকে

দুই স্লট ভর্তি, তাই *empty* = 0

কনস্ট্রাক্ট	মান	মালিক?	সেরা কাজে
সেমাফোর	পূর্ণসংখ্যা ≥ 0	**না** — যে কেউ V করতে পারে	গণনার রিসোর্স, ক্রম
মিউটেক্স	0 বা 1	**হ্যাঁ** — ধারকই আনলক করে	ক্রিটিক্যাল সেকশন
মনিটর	সুত্বক, ভেতরে একটি থ্রেড	স্বয়ংক্রিয়	ডেটা + প্রসিডিউর + কন্ডিশন ভ্যারিয়েবল

সমাধান ✍ তিনটি সেমাফোর N স্লটের বাফারের জন্য: ***mutex* = 1** (বাফারের কোডে একসময়ে একটি প্রসেস), ***empty* = N** (খালি স্লট), ***full* = 0** (ভর্তি স্লট)।

```
producer:                    consumer:
wait(empty);  wait(full);
wait(mutex);  wait(mutex);
  put item;      take item
signal(mutex); signal(mutex);
signal(full);  signal(empty);
```

বাইরের দুটি *wait*-এর **ক্রমই পুরো সমস্যা**: প্রোডিউসার মিউটেক্সের আগে খালি স্লট দাবি করে, কনজিউমার ভর্তি স্লট আগে। ক্রম উল্টালে দুজনেই মিউটেক্সের ওপর আটকে থাকে।

ট্রেস, বাফার ২	কাজ	*mutex*	*empty*	*full*	বাফার (*in* → *out*)
—	শুরু	1	2	0	খালি
১	P1 রাখে **x**	1	1	1	x
২	P2 রাখে **y**	1	0	2	x y
৩	P3 রাখতে চায় **z**	1	**0**	2	**P3 ব্লক**
৪	C1 নেয় **x**	1	1	1	y
৫	C2 নেয় **y**	1	2	0	খালি
৬	P3 জেগে **z** রাখে	1	1	1	z

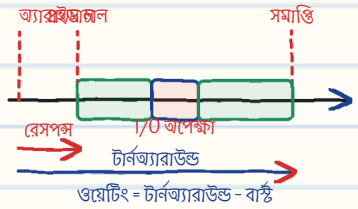
মনে রাখার কৌশল **"*empty* গোনে ফাঁকা জায়গা, *full* গোনে জিনিস, *mutex* গোনে অনুমতি, আর *empty* + *full* সবসময় = N।"** উপরের প্রতিটি সারিতে *empty* + *full* = ২; না হলে ট্রেসটাই ভুল।

পরীক্ষার ফাঁদ **P কমায়, V বাড়ায়** — সেমাফোর P-তে টাকা ঢালে। ৭ থেকে শুরু করে ১০টি P ও ৬টি V হলে শেষ মান $7 - 10 + 6 = 3$। যে মিউটেক্স তুমি ধরে না, তা আনলক করার অর্থ নেই।

৭ সিডিউলিংয়ের মাপ ও প্রিম্পশন সূত্র ★

সংজ্ঞা CPU শিডিউলিং ঠিক করে **কোন রেডি প্রসেস পরে চলবে** এবং **কতক্ষণ**। কোনো অ্যালগরিদম সব মাপে জেতে না, তাই সিডিউলিংকে বিচার করা হয় পাঁচটি মাপের বিপরীতে — কোন মাপটি গুরুত্বপূর্ণ তার ওপরই অ্যালগরিদম নির্ভর করে।

মাপ	সংজ্ঞা ও সূত্র	চাওয়া
CPU ব্যবহার	ব্যস্ত অংশ = 1 − **খালি অংশ**	যত **বেশি** তত ভালো
থ্রুপুট	একক সময়ে সম্পন্ন প্রসেস **সংখ্যা**	যত **বেশি** তত ভালো
টার্নঅ্যারাউন্ড টাইম	**সমাপ্তি − অ্যারাইভাল**, পুরো জীবন	**যত কম তত ভালো**
ওয়েটিং টাইম	**টার্নঅ্যারাউন্ড − CPU বার্স্ট**	**যত কম তত ভালো**
রেসপন্স টাইম	**প্রথম সাড়া − অ্যারাইভাল**	কম — **ইন্টারঅ্যাক্টিভ** ব্যবহারকারী যা অনুভব করে



CPU-তে না কাটানো সবই অপেক্ষা

বিষয়	নন-প্রিমপটিভ	প্রিমপটিভ
চলমান প্রসেস	শেষ বা ব্লক হওয়া পর্যন্ত CPU রাখে	**বাধা দেওয়া** যায় ও আবার সারিতে যায়
যা লাগে	কিছু বিশেষ নয়	টাইমার ইন্টারাপ্ট, OS সহায়তা
খরচ	কম — সুইচ কম	বেশি — প্রতি প্রিম্পশনে একটি
রেসপন্স টাইম	**খারাপ** হতে পারে: লম্বা কাজ সবাকে আটকায়	ভালো, q দিয়ে সীমাবদ্ধ
ফাঁদ	**ক্রিটিক্যাল সেকশনের** ভেতরে কখনো প্রিম্পট নয়	তা করলে অবস্থা নষ্ট হয়

মাল্টিপ্রসেসিং বনাম টাইম-শেয়ারিং — সিলেবাসের যে পার্থক্য চায়

মাল্টিপ্রোগ্রামিং: মেমোরিতে অনেক প্রোগ্রাম, CPU তাদের মধ্যে বদলায়

মাল্টিপ্রসেসিং: অনেক **প্রসেসর**, সত্যিকারের সমান্তরালতা

টাইম-শেয়ারিং: একটি CPU **সময় ভাগে** ভাগ করে — চিরায়ত টাইম-শেয়ারিং OS হলো রাউন্ড রবিন

মাল্টিটাস্কিং: একটি CPU-তে অনেক কাজ, টাইম-শেয়ারিংয়ের পেছনের ধারণা

রিয়েল-টাইম: থ্রুপুটের চেয়ে ডেডলাইন গুরুত্বপূর্ণ

ব্যাচ: থ্রুপুট বাড়ানো হয়, রেসপন্স উপেক্ষা করা হয়

পরীক্ষার ফাঁদ ওয়েটিং টাইম **টার্নঅ্যারাউন্ড নয়**, আর রেসপন্স টাইম **ওয়েটিং টাইম নয়**। বার্স্ট 10, অ্যারাইভাল 2, সমাপ্তি 20 হলে TAT = **18**, WT = **8**, আর রেসপন্স হিসাব হয় **প্রথম** পাঠানো থেকে।

মনে রাখার কৌশল "**T**র্নঅ্যার**া**উন্ড = অ্যার**া**ইভাল থেকে **স**মাপ্তি। **W**েটিং = টার্নঅ্যারাউন্ড − **B**ার্স্ট। **R**েসপন্স = অ্যারাইভাল থেকে প্রথম **R**ান।"

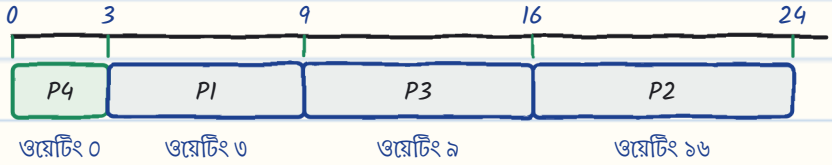
সমাধানকৃত উদাহরণ ✍ তিনটি সময়ই হিসাব করো একটি কাজ $t = 2$-এ আসে, প্রথমবার $t = 5$-এ পাঠানো হয়, $t = 9$-এ I/O-র জন্য ব্লক হয়, $t = 12$-এ আবার পাঠানো হয় এবং $t = 24$-এ শেষ হয়; মোট CPU বার্স্ট 12। **(ক)** টার্নঅ্যারাউন্ড = $24 - 2 =$ **22**। **(খ)** ওয়েটিং = $22 - 12 =$ **10**, ফাঁক দিয়ে যাচাই — $(5-2) + (12-9) = 6$ সারিতে থেকে, আর শেষ পাঠানোর ভেতরে 4। **(গ)** রেসপন্স = $5 - 2 =$ **3**। **(ঘ)** নন-প্রিমপটিভ হলে রেসপন্স 0 হতো — কিন্তু পাশের প্রসেসরা দিত।

৮ নন-প্রিমপটিভ: FCFS, SJF ও প্রায়োরিটি

সবচেয়ে বেশি আসে ★

সংজ্ঞা প্রসেস CPU রাখে যতক্ষণ না শেষ হয় বা ব্লক হয়। FCFS অ্যারাইভালের ক্রমে সেবা করে, SJF সবচেয়ে ছোট CPU বার্স্ট বাছে, প্রায়োরিটি শিডিউলিং সবচেয়ে উচ্চ অগ্রাধিকারের প্রসেস বাছে এবং টাই রাখে FCFS।

অ্যালগরিদম	নিয়ম	ভালো	খারাপ
FCFS	অ্যারাইভালের ক্রম	CPU সময়ে ন্যায্য, সহজ, স্টারভেশন নেই	**কনভয় ইফেক্ট**: একটি লম্বা কাজ প্রতিটি ছোট কাজ দেরি করায়
SJF	ছোট বার্স্ট আগে	**ন্যূনতম গড় অপেক্ষা**, প্রমাণযোগ্যভাবে সেরা	বার্স্ট আগেই জানা লাগে; লম্বা কাজ **স্টার্ভ** হতে পারে
SRTF (প্রিমপটিভ)	ছোট *বাকি* সময় আগে	গড় অপেক্ষাতেও সেরা	বেশি সুইচ; বার্স্টের অনুমান লাগে
প্রায়োরিটি	বেশি অগ্রাধিকার আগে	গুরুত্ব বোঝায়; জরুরি কাজে দ্রুত সাড়া	কম অগ্রাধিকারের প্রসেস **স্টার্ভ** হয়
রাউন্ড রবিন (প্রিমপটিভ)	পালা করে একটি **টাইম কোয়ান্টাম**	ন্যায্য CPU ভাগ, সীমিত রেসপন্স	কোয়ান্টাম ছোট হলে সুইচের খরচ



বার্স্ট 6, 8, 7, 3 ms-এ SJF: P4 0–3, P1 3–9, P3 9–16, P2 16–24

সমাধানকৃত উদাহরণ ১ ✍ FCFS বার্স্ট **24, 3, 3**, সবাই $t = 0$-এ। P1 0–24, P2 24–27, P3 27–30। ওয়েটিং 0, 24, 27 → গড় 51/3 = **17 ms**। দুটো 3 ms কাজ 24 ms কাজের পেছনে বসে ছিল — এটাই কনভয়।

সমাধানকৃত উদাহরণ ২ ✍ SJF বার্স্ট P1 = 6, P2 = 8, P3 = 7, P4 = 3। ক্রম P4 0–3, P1 3–9, P3 9–16, P2 16–24। ওয়েটিং 0, 3, 9, 16 → গড় 28/4 = **7 ms**। একই সেটে FCFS দিলে 0, 6, 14, 21 → **10.25 ms**।

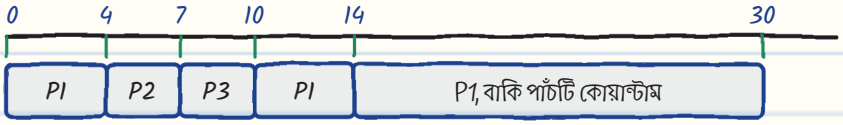
পরীক্ষার ফাঁদ SJF **গড় অপেক্ষার** জন্য সেরা — টার্নঅ্যারাউন্ড বা রেসপন্সের জন্য নয়, ওজনযুক্ত সমন্বয়ের জন্যও নয়। বার্স্ট **আগে থেকে জানা** লাগে; বাস্তবে OS ইতিহাস থেকে অনুমান করে, তাই SRTF আসলে "শর্টেস্ট রিমেইনিং টাইম *নেক্সট*"।

মনে রাখার কৌশল SJF-র জন্য "**ছোট আগে, লম্বা পরে**"; FCFS-এর রোগ **কনভয়**; প্রায়োরিটির রোগ **স্টারভেশন**, আর তার ওষুধ এজিং — অপেক্ষার সময় বাড়লে কার্যকর অগ্রাধিকারও বাড়ে।

এজিং ঠিকভাবে প্রতিটি অপেক্ষার ধাপ পর প্রসেসের কার্যকর অগ্রাধিকার বাড়ানো যায়, অথবা "অনেক দিন ধরে অপেক্ষা করেছে" ধরে সেটিকে উন্নীত করা যায়। **এজিং কেবল স্টারভেশন ঠিকায়**: ডেডলকে এর কোনো প্রভাব নেই, কারণ সেখানে কিছুই এগোয় না এবং অগ্রাধিকার পুনরায় মূল্যায়ন হয় না। এজিং ছাড়া খাঁটি প্রায়োরিটি সারি প্রায় সব বাস্তব নকশায় গ্রহণ করা হয় না।

৯ প্রিমপটিভ: SRTF ও রাউন্ড রবিন গ্যান্ট চার্ট ★

সংজ্ঞা SRTF নতুন আসা কোনো কাজের বাকি সময় ছোট হলেই চলমান প্রসেসকে প্রিম্পট করে। রাউন্ড রবিন (Round Robin) একটি গোলাকার **রেডি সারি** রাখে এবং প্রতি **টাইম কোয়ান্টাম q** -এর পর প্রিম্পট করে; সময় অবশিষ্ট থাকলে প্রসেসটি সারির **পিছনে** যায়।



ওয়েটিং ৬ ওয়েটিং ৪ওয়েটিং ৭

রাউন্ড রবিন, $q = 4$ ms; তিনটিই 0-এ এসেছে, বার্স্ট 24 / 3 / 3 — ওয়েটিং = সমাপ্তি − অ্যারাইভাল − বার্স্ট

সমাধানকৃত উদাহরণ ১ ✍ রাউন্ড রবিন, $q = 4$

P1 = 24, P2 = 3, P3 = 3, সবাই 0-এ। গ্যান্ট: P1 0–4, P2 4–7, P3 7–10, এরপর P1 একাই 10–30। ওয়েটিং = সমাপ্তি − বার্স্ট: P1 = 30 − 24 = 6, P2 = 7 − 3 = 4, P3 = 10 − 3 = 7 → গড় 17/3 = **5.67 ms**। এই সেটে SJF দিলে 0, 3, 6 → **3 ms**; FCFS 17 ms।

সমাধানকৃত উদাহরণ ২ ✍ SRTF, ভিন্ন সময়ে অ্যারাইভাল

P1 (0, 8), P2 (1, 4), P3 (2, 9), P4 (3, 5)। $t = 1$-এ P2 (4) হারায় P1-এর বাকি 7-কে → প্রিম্পট; $t = 5$-এ P4 (5) হারায় P1 (7) ও P3 (9)-কে; $t = 10$-এ P1 (7) হারায় P3 (9)-কে। গ্যান্ট: **P1 0–1 | P2 1–5 | P4 5–10 | P1 10–17 | P3 17–26**। সমাপ্তি 17, 5, 26, 10 → টার্নঅ্যারাউন্ড 17, 4, 24, 7 → গড় **13 ms** ওয়েটিং 9, 0, 15, 2 → গড় **6.5 ms**।

টাইম কোয়ান্টাম q	আচরণ
q বড় ($q \geq$ বার্স্ট)	বাস্তবে প্রিম্পশন হয় না → **RR আসলে FCFS হয়ে যায়**
q ছোট	অনেক কনটেক্সট সুইচ; বেশি সময় সোয়াপেই যায়, থ্রুপুট কমে
গড় বার্স্টের সামান্য বেশি	সাধারণ পছন্দ — ইন্টারঅ্যাক্টিভ থাকার মতো দ্রুত, আবার অধিকাংশ কাজ কয়েক পালায় শেষ
সবচেয়ে খারাপ রেসপন্স	nটি রেডি প্রসেসে সীমাবদ্ধ $(n - 1) \times q$

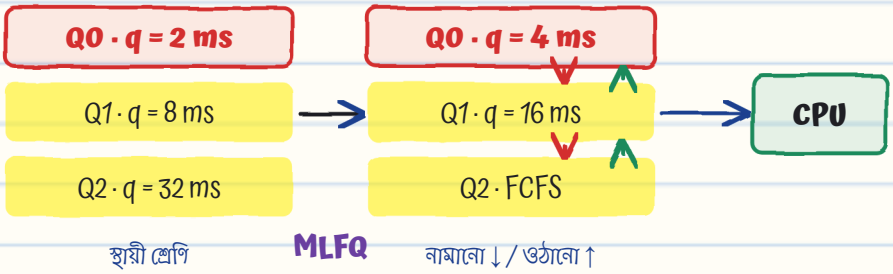
মনে রাখার কৌশল "কোয়ান্টাম আসলে **অপেক্ষার একটা প্রতিশ্রুতি**: ছোট q ন্যায্যতা ও দ্রুত প্রথম সাড়ার কথা বলে এবং সেজি খরচে চোখায়; বড় q দক্ষতার কথা বলে এবং রেসপন্স টাইমে ভরি করে।"

পরীক্ষার ফাঁদ (i) RR-তে ওয়েটিং টাইম হলো **সমাপ্তি − অ্যারাইভাল − বার্স্ট**, প্রতি কোয়ান্টাম গুনে "সারিতে থাকা সময়" নয়। (ii) কোয়ান্টামের ভেতরেই শেষ হয়ে গেলে প্রসেস **আর অপেক্ষা করে না**। (iii) RR **প্রিমপটিভ**; "স্লাইসে শেষ পর্যন্ত চালানো" বলে যা লেখা সেটা FCFS।

আবার থ্রেড বনাম প্রসেস শিডিউলিং প্রতিটি প্রসেসে একটি থ্রেড থাকলে RR প্রতিটি **প্রসেসকে** পালা দেয় — একটি প্রসেসের পঞ্চাশ থ্রেড থাকলে এটি অন্যায়। আধুনিক কার্নেল তাই **থ্রেড** শিডিউল করে, যাতে প্রতিটি চালনোগার্য থ্রেড নিজের পালা পায়।

১০ মাল্টিলেভেল সারি, ফিডব্যাক ও রিয়েল-টাইম অগ্রসর ★

সংজ্ঞা একটি মাল্টিলেভেল সারি স্থায়ী শ্রেণি রাখে — সিস্টেম প্রসেস ছোট কোয়ান্টাম ও রাউন্ড রবিনে, ইউজার প্রসেস FCFS বা SJF-তে। একটি মাল্টিলেভেল ফিডব্যাক সারি (MLFQ) সরতে দেয়: পুরো কোয়ান্টাম খাটিয়ে ফেললে প্রসেস নিচে নামে, বেশি অপেক্ষা করলে উপরে ওঠে।



পুরো পার্থক্য: প্রসেস কি **সারি বদলাতে** পারে?

বিষয়	মাল্টিলেভেল সারি	মাল্টিলেভেল ফিডব্যাক সারি
চলাচল	**নেই** — শ্রেণি স্থায়ী	প্রসেস উপরে-নিচে যায়
প্রতি সারির কোয়ান্টাম	প্রতি শ্রেণিতে একটি	অগ্রাধিকারা কমলে বাড়ে
শক্তি	সহজ ও সস্তা; প্রতি শ্রেণিতে আলাদা ভর্তি নীতি	**ইন্টারঅ্যাক্টিভ** ও **CPU-বাউন্ড** কাজ নিজে থেকেই আলাদা করে
দুর্বলতা	নিচের শ্রেণি **স্টার্ভ** হতে পারে	স্টারভেশন এড়াতে প্রমোশন নিয়ম লাগে

রিয়েল-টাইম নীতি	অগ্রাধিকারার নিয়ম
রেট মনোটনিক	**স্থির**: ছোট পিরিয়ড → বেশি অগ্রাধিকারা
EDF	**পরিবর্তনশীল**: সবার আগে ডেডলাইন আছে যার

হার্ড রিয়েল-টাইম এক লাইনে এক প্রসেসরে nটি পিরিয়ডিক কাজের জন্য প্রতিটির $C \le T$ লাগে **এবং** মোট ব্যবহার $\le n(2^{(1/n)} - 1)$: ১ কাজ ১০০ %, ২ কাজ ৮৩ %, ৩ কাজ ৭৮ %।

পরীক্ষার ফাঁদ **EDF সেরা** — এটি কেবল তখনই ডেডলাইন বাদ দেয় যখন কোনো নির্ভেজ্ঞ সূচি-ই নেই। রেট মনোটনিক কেবল **স্থির** অগ্রাধিকারায় সেরা এবং ৭৬ %-এর বেশি ব্যবহারে EDF-এর কাছে হেরে যায়।

মনে রাখার কৌশল "**স্থায়ী শ্রেণি সরে না; ফিডব্যাক সারি শিডিউলারকে শেখায়।**" আর "**রেট স্থির, ডেডলাইন** প্রতি মুহূর্তে আবার পড়া হয়।"

পরীক্ষার ফাঁদ দীর্ঘ শিডিউলার খালি না থাকলে কেবল **সর্বোচ্চ** সারিতেই সেবা দেয়, তাই নিচের শ্রেণি অনির্দিষ্টকাল অপেক্ষায় থাকে — যা **এজিং** বা প্রমোশনই ঠিক করে।

সমাধানকৃত উদাহরণ ✍ দুটি পিরিয়ডিক কাজ একটি কন্ট্রোল লুপ প্রতি 40 ms-এ 10 ms চায়; একটি লগার প্রতি 100 ms-এ 5 ms চায়। **(ক)** ব্যবহার = 10/40 + 5/100 = 0.25 + 0.05 = **0.30**। **(খ)** দুটি কাজ, তাই রেট মনোটনিক সীমা $2(2^{(1/2)} - 1) \approx$ **0.828**; 0.30 < 0.828, তাই সেটটি **রেট মনোটনিকে চালানোর উপযোগী**। **(গ)** EDF একই ডেডলাইন মেলায় অনেক বেশি ভার নিতে পারে। **(ঘ)** লুপের পিরিয়ড 12 ms হলে ব্যবহার ≈ 0.883 > 0.828, তখন রেট মনোটনিক আর নিশ্চয়তা দিতে পারে না, EDF ব্যবহার করতে হবে।

রাতের-পরীক্ষার পাতা

তথ্য-পত্র

প্রোগ্রাম = নিষ্ক্রিয় ফাইল · প্রসেস = চলমান
PID চিহ্নিত করে; পরে পুনর্ব্যবহার
new → ready → running → waiting → terminated
দীর্ঘমেয়াদি = জব সারি · মধ্যবর্তী = সোয়াপ
থ্রেডরা অ্যাড্রেস স্পেস ভাগ করে
ইউজার-লেভেল পুরো প্রসেস থামায়
many-to-one · one-to-one · many-to-many
IPC: shm, pipe, FIFO, queue, socket, signal
সেমাফোর P = −1 (ব্লক হতে পারে), V = +1
empty + full = বাফারের আকার N
TAT = সমাপ্তি − অ্যারাইভাল
SJF = ন্যূনতম গড় অপেক্ষা
RR: q বড় → FCFS, q ছোট → সুইচ বেশি
MLFQ CPU-বাউন্ডকে নামায়, অপেক্ষাকারীকে ওঠায়

প্রসেস = কোড, ডেটা, PC, রেজিস্টার
PCB: স্টেট, PC, রেজিস্টার, শিডিউলিং, মেমোরি, হিসাব
রেডি = CPU চায়, ওয়েটিং = ঘটনা চায়

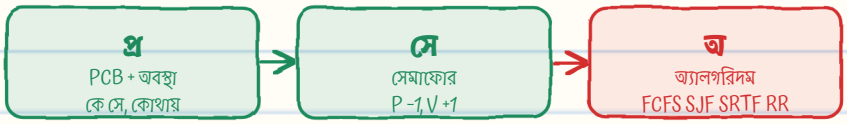
স্বল্পমেয়াদি = CPU শিডিউলার
নিজস্ব স্ট্যাক ও রেজিস্টার
কার্নেল-লেভেল একটি থ্রেড থামায়
কার্নেল শুধু কার্নেল থ্রেড শিডিউল করে
অ্যানোনিমাস পাইপ = সম্পর্কিত প্রসেস
মিউটেক্সের মালিক আছে, সেমাফোরের নেই
প্রোডিউসার empty আগে, কনজিউমার full আগে
WT = TAT − বার্স্ট · রেসপন্স = প্রথম চাল − অ্যারাইভাল
FCFS = কনভয় · প্রায়োরিটি = স্টারভেশন
RR-এ সবচেয়ে খারাপ রেসপন্স = (n − 1)q
EDF সেরা · RM সীমা n(2^(1/n) − 1)

→ MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. পাঁচটি অবস্থা, চার নয় — **new** আলাদা
2. রেডি মানে শুধু **CPU**-র অপেক্ষা
3. তিন শিডিউলার: দীর্ঘ, মধ্যবর্তী, স্বল্পমেয়াদি
4. PCB একটি **কার্নেল** কাঠামো
5. থ্রেড স্ট্যাক নয়, অ্যাড্রেস স্পেস ভাগ করে
6. many-to-one: একটি ব্লকিং কল **সব** থ্রেড থামায়
7. শেয়ার্ড মেমোরি দ্রুততম, কিন্তু **নিজের** লক লাগে
8. P কমায়, V বাড়ায়
9. SJF **গড় অপেক্ষায়** সেরা
10. FCFS-এর রোগ **কনভয়**
11. q বাড়লে RR → FCFS হয়ে যায়
12. EDF সেরা; RM সীমা n(2^(1/n) − 1)

সবচেয়ে বড় ফাঁদ রেডি বনাম ওয়েটিং · WT ≠ TAT, রেসপন্স ≠ WT · SJF কেবল গড় **অপেক্ষায়** সেরা · নতুন প্রসেস **রেডি** সারিতে যায়, running-এ নয় · অ্যানোনিমাস পাইপ অসম্পর্কিত প্রসেস অতিক্রম করতে পারে না · শেয়ার্ড মেমোরি **কোনো** মিউচুয়াল এক্সক্লুশন দেয় না · যে মিউটেক্স তুমি ধরো না তা আনলক করা অর্থহীন · এজিং স্টারভেশন ঠিকায়, ডেডলক কখনো নয় · FCFS-এ স্টারভেশন হয় না, প্রায়োরিটি ও SJF-তে হয় · দুই কাজের রিয়েল-টাইম সীমা ৮৩ %, ১০০ % নয়।

সব কৌশল একসাথে "প্রোগ্রাম রেসিপি, প্রসেস তৈরি হওয়া খাবার আর পাশে রাখা টিকিট" · "রেডি CPU চায়, ওয়েটিং অন্য কিছু চায়" · "জন্ম রেডি, মৃত্যু শেষ" · "থ্রেড ভাগ করে, কপি করে না" · "কপি = মেসেজ পাসিং, ম্যাপ = শেয়ার্ড মেমোরি, নামযুক্ত = অসম্পর্কিত" · "সেমাফোর P-তে টাকা ঢালে" · "empty + full সবসময় = N" · "ছোট আগে, লম্বা পরে" · "কোয়ান্টাম অপেক্ষার প্রতিশ্রুতি" · "স্থায়ী শ্রেণি সরে না; ফিডব্যাক সারি শেখায়" · "রেট স্থির, ডেডলাইন প্রতি মুহূর্তে নতুন করে পড়া হয়"।



প্রতিটি সিডিউলিং প্রশ্ন = একটি গ্যান্ট চার্ট আর একটি বিয়োগ

প্রসেস, সিঙ্ক্রোনাইজেশন, সিডিউলিং — এই লেসনের তিন অর্ধেক

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

আগে চেষ্টা করো, তারপর দেখো

১. মেমোরিতে, শুধু CPU-এর অপেক্ষায়:
ক) New খ) Waiting
গ) Terminated ঘ) Ready

২. নতুন তৈরি প্রসেস যায়:
ক) running অবস্থায় খ) ডিভাইস সারিতে
গ) waiting অবস্থায় ঘ) রেডি সারিতে

৩. PCB-র কোনটি **ঘর নয়**?
ক) প্রোগ্রাম কাউন্টার খ) CPU রেজিস্টার
গ) শেষ ডিস্ক সেক্টর ঘ) প্রসেস স্টেট

৪. শুধু CPU-এর দরকার এমন একটি ব্লক করা প্রসেড:
ক) প্রিম্পট হয় খ) টার্মিনেট হয়
গ) পাঠানো হয় ঘ) ভর্তি হয়

৫. রাউন্ড রবিন মূলত কোনের জন্য:
ক) ব্যাচ খ) টাইম-শেয়ারিং
গ) সিঙ্গেল-টাস্কিং ঘ) স্পুলিং

৬. একই প্রসেসের থ্রেডগুলো ভাগ করে:
ক) স্ট্যাক খ) অ্যাড্রেস স্পেস
গ) রেজিস্টার ঘ) PC-র মান

৭. S = 7, তারপর ১০টি P ও ৬টি V:
ক) ৩ খ) ১
গ) ১৩ ঘ) ০

৮. বার্স্ট 24, 3, 3, t = 0; FCFS গড় অপেক্ষা:
ক) 3 ms খ) 13 ms
গ) 17 ms ঘ) 27 ms

৯. একই বার্স্ট, q = 4, RR গড় অপেক্ষা:
ক) 4.00 ms খ) 5.67 ms
গ) 7.33 ms ঘ) 17.00 ms

১০. প্যারেন্ট আগে মারা গেলে শিশু:
ক) জম্বি খ) অরফান
গ) ডেমন ঘ) কার্নেল থ্রেড

উত্তরমালা → ১-ঘ · ২-ঘ · ৩-গ · ৪-গ · ৫-খ · ৬-খ · ৭-ক · ৮-গ · ৯-খ · ১০-খ

লিখিত প্রশ্ন প্রসেস কী, আর প্রসেস কন্ট্রোল ব্লকে কী থাকে? পাঁচ-অবস্থার ডায়াগ্রামটি সাতটি তিরসহ আঁকো এবং মেমোরিতে থেকে শুধু CPU-এর অপেক্ষায় থাকা প্রসেস কোন সারিতে পড়ে। বার্স্ট P1 = 24, P2 = 3, P3 = 3 ms সবাই t = 0-এ এসেছে: FCFS-এ এবং q = 4 ms রাউন্ড রবিনে গড় ওয়েটিং টাইম বের করো, আর গড় অপেক্ষার জন্য কোন অ্যালগরিদম সেরা তা নামাও। ১০ নম্বর

১ **প্রসেস ও PCB** — মেমোরিতে তুলে চলা একটি প্রোগ্রাম, সঙ্গে কোড, ডেটা, প্রোগ্রাম কাউন্টার, রেজিস্টার ও খোলা ফাইল। PCB = প্রসেস স্টেট, প্রোগ্রাম কাউন্টার, CPU রেজিস্টার, CPU শিডিউলিং তথ্য, মেমোরি ম্যানেজমেন্ট তথ্য, অ্যাকাউন্টিং তথ্য। 3

২ **ডায়াগ্রাম** — new → ready (ভর্তি); ready → running (পাঠানো); running → ready (ইন্টারাপ্ট বা স্লাইস শেষ); running → waiting (I/O বা ঘটনার অপেক্ষা); waiting → ready (সমাপ্তি); running → terminated; new → terminated (ভর্তি বাতিল)। শুধু CPU-এর অপেক্ষা মানে **রেডি সারি**, ডিভাইস সারি নয়। 3

৩ **FCFS** — P1 0–24, P2 24–27, P3 27–30। ওয়েটিং 0, 24, 27 → গড় 51/3 = **17 ms**। 2

৪ **RR, q = 4** — P1 0–4, P2 4–7, P3 7–10, এরপর P1 একাই 10–30। ওয়েটিং = সমাপ্তি − বার্স্ট: 6, 4, 7 → গড় 17/3 = **5.67 ms**। গড় অপেক্ষায় সেরা: **SJF** (প্রিম্পটিভ SRTF) — 0, 3, 6 → **3 ms**। 2

→ পরীক্ষার আগে ✓

- ✓ পাঁচটি অবস্থা ও সাতটি রূপান্তর নামাতে পারি
- ✓ PCB-র ঘরগুলো মুখস্থ ছাড়া বলতে পারি
- ✓ FCFS, SJF ও RR-এর গ্যান্ট চার্ট আঁকতে পারি
- ✓ প্রোডিউসার–কনজিউমারে সেমাফোর ট্রেস করতে পারি